

INTERVIEW PREPARATION SERIES

Deque & Circular Linked Lists

LeetCode Practice Set — 10 Problems

This practice set covers **10 LeetCode problems** testing deque design, monotonic deque optimization, circular deque implementation, prefix-sum deque combinations, DP with deque, Floyd's cycle detection, sorted circular insertion, and implicit cycle detection in arrays. Each problem includes a 4-tier breakdown. Solve in any language — the concepts are universal.

PROBLEM 01

Design Front Middle Back Queue Medium

TIER 1 — UNDERSTAND

Design a queue supporting push/pop at front, middle, and back. Tests advanced deque mechanics.

TIER 2 — APPROACH

Two deques: split elements across a left and right deque. Keep them balanced (sizes differ by at most 1). Middle operations interact with the junction.

TIER 3 — ALGORITHM SKETCH

```
from collections import deque
class FrontMiddleBackQueue:
    def __init__(self): self.left=deque(); self.right=deque()
    def _balance(self):
        while len(self.left)>len(self.right):
            self.right.appendleft(self.left.pop())
        while len(self.left)<len(self.right)-1:
            self.left.append(self.right.popleft())
    def pushFront(self,v): self.left.appendleft(v); self._balance()
    def pushMiddle(self,v): self.left.append(v); self._balance()
    def pushBack(self,v): self.right.append(v); self._balance()
    def popFront(self):
        if not self.left and not self.right: return -1
        v=self.left.popleft() if self.left else self.right.popleft()
        self._balance(); return v
```

TIER 4 — COMPLEXITY

Time: $O(1)$ amortized all operations | **Space:** $O(n)$

PROBLEM 02

Sliding Window Maximum Hard**TIER 1 — UNDERSTAND**

Return the max in each sliding window of size k. The classic monotonic deque problem.

TIER 2 — APPROACH

Decreasing deque of indices: front = window max. Remove expired from front, smaller from back.

TIER 3 — ALGORITHM SKETCH

```
def maxSlidingWindow(nums, k):
    dq, res = deque(), []
    for i in range(len(nums)):
        while dq and dq[0] < i - k + 1: dq.popleft()
        while dq and nums[dq[-1]] <= nums[i]: dq.pop()
        dq.append(i)
        if i >= k - 1: res.append(nums[dq[0]])
    return res
```

TIER 4 — COMPLEXITY

Time: $O(n)$ | **Space:** $O(k)$

PROBLEM 03

Design Circular Deque Medium**TIER 1 — UNDERSTAND**

Design a fixed-capacity circular deque with insertFront, insertLast, deleteFront, deleteLast, getFront, getRear, isEmpty, isFull.

TIER 2 — APPROACH

Circular array with front pointer and size counter. Compute rear as $(\text{front} + \text{size}) \% \text{cap}$.

TIER 3 — ALGORITHM SKETCH

```
class MyCircularDeque:
    def __init__(self, k): self.d = [0] * k; self.f = 0; self.sz = 0; self.cap = k
    def insertFront(self, v):
        if self.sz == self.cap: return False
        self.f = (self.f - 1) % self.cap; self.d[self.f] = v; self.sz += 1; return
```

```
True
def insertLast(self,v):
    if self.sz==self.cap: return False
    self.d[(self.f+self.sz)%self.cap]=v; self.sz+=1; return True
def deleteFront(self):
    if self.sz==0: return False
    self.f=(self.f+1)%self.cap; self.sz-=1; return True
def deleteLast(self):
    if self.sz==0: return False
    self.sz-=1; return True
def getFront(self): return -1 if self.sz==0 else self.d[self.f]
def getRear(self): return -1 if self.sz==0 else self.d[(self.f+self.sz-1)%self.cap]
```

TIER 4 — COMPLEXITY**Time:** O(1) all operations | **Space:** O(k)**PROBLEM 04****Shortest Subarray with Sum at Least K** Hard**TIER 1 — UNDERSTAND**

Find the shortest subarray with sum $\geq k$. Array may contain negative numbers, so sliding window alone fails. Tests monotonic deque with prefix sums.

TIER 2 — APPROACH

Compute prefix sums. Use a deque to find the smallest $j-i$ where $\text{prefix}[j]-\text{prefix}[i] \geq k$.

TIER 3 — ALGORITHM SKETCH

```
def shortestSubarray(nums, k):
    n=len(nums); prefix=[0]*(n+1)
    for i in range(n): prefix[i+1]=prefix[i]+nums[i]
    dq=deque(); ans=n+1
    for j in range(n+1):
        while dq and prefix[j]-prefix[dq[0]]>=k:
            ans=min(ans,j-dq.popleft())
        while dq and prefix[j]<=prefix[dq[-1]]:
            dq.pop()
        dq.append(j)
    return ans if ans<=n else -1
```

TIER 4 — COMPLEXITY**Time:** O(n) | **Space:** O(n)

PROBLEM 05

Constrained Subsequence Sum Hard

TIER 1 — UNDERSTAND

Find the max sum of a non-empty subsequence where consecutive elements are at most k indices apart. Tests monotonic deque optimization of DP.

TIER 2 — APPROACH

DP: $dp[i] = nums[i] + \max(dp[j])$ for j in $[i-k, i-1]$. Use a monotonic deque to track the max dp value in the window.

TIER 3 — ALGORITHM SKETCH

```
def constrainedSubsetSum(nums, k):
    n=len(nums); dp=nums[:]
    dq=deque([0])
    for i in range(1,n):
        while dq and dq[0]<i-k: dq.popleft()
        dp[i]=max(dp[i], dp[dq[0]]+nums[i])
        while dq and dp[dq[-1]]<=dp[i]: dq.pop()
        dq.append(i)
    return max(dp)
```

TIER 4 — COMPLEXITY

Time: $O(n)$ | **Space:** $O(n)$

PROBLEM 06

Linked List Cycle Easy

TIER 1 — UNDERSTAND

Determine if a linked list has a cycle. Tests Floyd's Tortoise and Hare in $O(1)$ space.

TIER 2 — APPROACH

Slow pointer (1 step) and fast pointer (2 steps). If they meet, cycle exists.

TIER 3 — ALGORITHM SKETCH

```
def hasCycle(head):
    slow=fast=head
    while fast and fast.next:
        slow=slow.next; fast=fast.next.next
        if slow==fast: return True
    return False
```

TIER 4 — COMPLEXITY

Time: $O(n)$ | **Space:** $O(1)$

PROBLEM 07

Linked List Cycle II Medium

TIER 1 — UNDERSTAND

Find the node where the cycle begins. Tests the mathematical proof behind Floyd's algorithm Phase 2.

TIER 2 — APPROACH

After detection, reset one pointer to head. Advance both by 1 until they meet at cycle start.

TIER 3 — ALGORITHM SKETCH

```
def detectCycle(head):
    slow=fast=head
    while fast and fast.next:
        slow=slow.next; fast=fast.next.next
        if slow==fast:
            slow=head
            while slow!=fast: slow=slow.next; fast=fast.next
            return slow
    return None
```

TIER 4 — COMPLEXITY

Time: $O(n)$ | **Space:** $O(1)$

PROBLEM 08

Insert into a Sorted Circular Linked List Medium

TIER 1 — UNDERSTAND

Insert a value into a sorted circular singly linked list, maintaining sorted order. Handles wrap-around at max→min boundary.

TIER 2 — APPROACH

Three cases: fits between two nodes, is new max/min (insert at boundary), or all values are equal.

TIER 3 — ALGORITHM SKETCH

```
def insert(head, val):
    node=Node(val)
```

```
if not head: node.next=node; return node
curr=head
while True:
    if curr.val<=val<=curr.next.val: break
    if curr.val>curr.next.val:
        if val>=curr.val or val<=curr.next.val: break
    curr=curr.next
    if curr==head: break
node.next=curr.next; curr.next=node
return head
```

TIER 4 — COMPLEXITY**Time:** $O(n)$ | **Space:** $O(1)$ **PROBLEM 09****Find the Duplicate Number** Medium**TIER 1 — UNDERSTAND**

Array of $n+1$ integers in range $[1, n]$. Find the duplicate without modifying the array. $O(1)$ space. Tests Floyd's algorithm on an implicit linked list.

TIER 2 — APPROACH

Treat the array as a linked list: index $\rightarrow$ nums[index]. A duplicate means two indices point to the same value, creating a cycle. Find the cycle start.

TIER 3 — ALGORITHM SKETCH

```
def findDuplicate(nums):
    slow=fast=nums[0]
    while True:
        slow=nums[slow]; fast=nums[nums[fast]]
        if slow==fast: break
    slow=nums[0]
    while slow!=fast:
        slow=nums[slow]; fast=nums[fast]
    return slow
```

TIER 4 — COMPLEXITY**Time:** $O(n)$ | **Space:** $O(1)$ **PROBLEM 10****Circular Array Loop** Medium

TIER 1 — UNDERSTAND

Detect a cycle in a circular array where each element indicates how many steps to move. Cycle must be all-positive or all-negative direction and length > 1.

TIER 2 — APPROACH

For each index, use Floyd's slow/fast on the circular array. Validate that the cycle has consistent direction and length > 1.

TIER 3 — ALGORITHM SKETCH

```
def circularArrayLoop(nums):
    n=len(nums)
    def nxt(i): return (i+nums[i])%n
    for i in range(n):
        if nums[i]==0: continue
        slow=fast=i
        while nums[nxt(slow)]*nums[i]>0 and nums[nxt(nxt(fast))]*nums[i]>0
        and nums[nxt(fast)]*nums[i]>0:
            slow=nxt(slow); fast=nxt(nxt(fast))
            if slow==fast:
                if slow==nxt(slow): break # length 1
                return True
        j=i
        while nums[j]*nums[i]>0: nums[j],j=0,nxt(j)
    return False
```

TIER 4 — COMPLEXITY

Time: O(n) | **Space:** O(1)

SUMMARY TABLE

#	Problem	Difficulty	Key Pattern	Topic
1	Front Middle Back Queue	Medium	Dual Deque Balance	Deque Design
2	Sliding Window Maximum	Hard	Monotonic Deque	Deque
3	Design Circular Deque	Medium	Circular Array	Deque
4	Shortest Subarray $\geq K$	Hard	Deque + Prefix Sum	Deque
5	Constrained Subseq Sum	Hard	Deque DP	Deque

6	Linked List Cycle	Easy	Floyd's Algorithm	Circular List
7	Linked List Cycle II	Medium	Cycle Start	Circular List
8	Insert Sorted Circular	Medium	Boundary Cases	Circular List
9	Find Duplicate Number	Medium	Implicit Cycle	Floyd's
10	Circular Array Loop	Medium	Direction Validation	Circular Array